

Documento de persistencia - SmartLogix

Fecha de revision: 06 de junio de 2026

Este documento explica como SmartLogix implementa persistencia con Spring Data JPA, Flyway y base de datos por servicio, manteniendo independencia entre microservicios.

Principio aplicado

Cada microservicio mantiene su propia base de datos y accede solo a sus tablas mediante repositories propios. Cuando un servicio necesita informacion de otro modulo, usa clientes REST y DTOs; no consulta tablas ajenas ni comparte entidades JPA.

Mapa de persistencia por servicio

Servicio	Base de datos	Entidades principales	Repositories	Migracion Flyway
auth-service	smartlogix_auth	User, Role	UserRepository, RoleRepository	auth-service/src/main/resources/db/migration/V1__create_auth_schema.sql
inventario	smartlogix	Product, Inventory	ProductRepository, InventoryRepository	inventario/src/main/resources/db/migration/V1__create_inventory_schema.sql
pedidos	smartlogix_orders	Order, OrderItem	OrderRepository	pedidos/src/main/resources/db/migration/V1__create_orders_schema.sql
envios	smartlogix_shipping	Shipment, ShipmentEvent	ShipmentRepository, ShipmentEventRepository	envios/src/main/resources/db/migration/V1__create_shipping_schema.sql

JPA y Spring Data

Las entidades JPA representan el modelo persistente interno de cada microservicio. Los controllers no exponen entidades directamente; transforman entrada y salida mediante DTOs. Los repositories encapsulan el acceso a datos y los services concentran reglas de negocio, transacciones y validaciones.

La configuracion usa `spring.jpa.hibernate.ddl-auto=validate`. Esto evita que Hibernate cree o altere tablas automaticamente en ejecucion normal; el esquema se valida contra las migraciones aplicadas.

Flyway

Flyway se habilita con `spring.flyway.enabled=true` y usa `classpath:db/migration`. Las migraciones V1 crean las tablas, indices, claves unicas y restricciones CHECK/FOREIGN KEY de cada servicio. Esto permite versionar el esquema y reproducirlo por ambiente.

Separacion de bases

auth-service administra usuarios, roles y JWT; inventario administra productos y stock; pedidos administra ordenes e items; envios administra despachos y eventos. Pedidos consulta inventario y auth-service por HTTP; envios consulta pedidos por HTTP. Esta separacion conserva el patron Database per Service.

Configuracion actual relevante

Servicio	Propiedad principal	Valor por defecto
auth-service	<code>spring.datasource.url</code>	<code>jdbc:mysql://localhost:3306/smartlogix_auth</code>
inventario	<code>spring.datasource.url</code>	<code>jdbc:mysql://localhost:3306/smartlogix</code>
pedidos	<code>spring.datasource.url</code>	<code>jdbc:mysql://localhost:3306/smartlogix_orders</code>
envios	<code>spring.datasource.url</code>	<code>jdbc:mysql://localhost:3306/smartlogix_shipping</code>
Todos	<code>spring.jpa.hibernate.ddl-auto</code>	<code>validate</code>

Servicio	Propiedad principal	Valor por defecto
Todos	spring.flyway.locations	classpath:db/migration

Buenas practicas observadas

Se usan UUID persistidos como CHAR para compatibilidad con MySQL, indices sobre campos de busqueda frecuentes, restricciones de unicidad para email/SKU/tracking y tablas de relacion/eventos donde corresponde. Las pruebas usan H2 o mocks para validar comportamiento sin depender de MySQL real.

Ajuste recomendado

La base del servicio inventario aparece como smartlogix. Funcionalmente puede ser una base propia si solo la usa inventario; para mayor claridad documental y operativa conviene renombrarla a smartlogix_inventory en ambientes nuevos, actualizando DB_URL y scripts de provisionamiento.